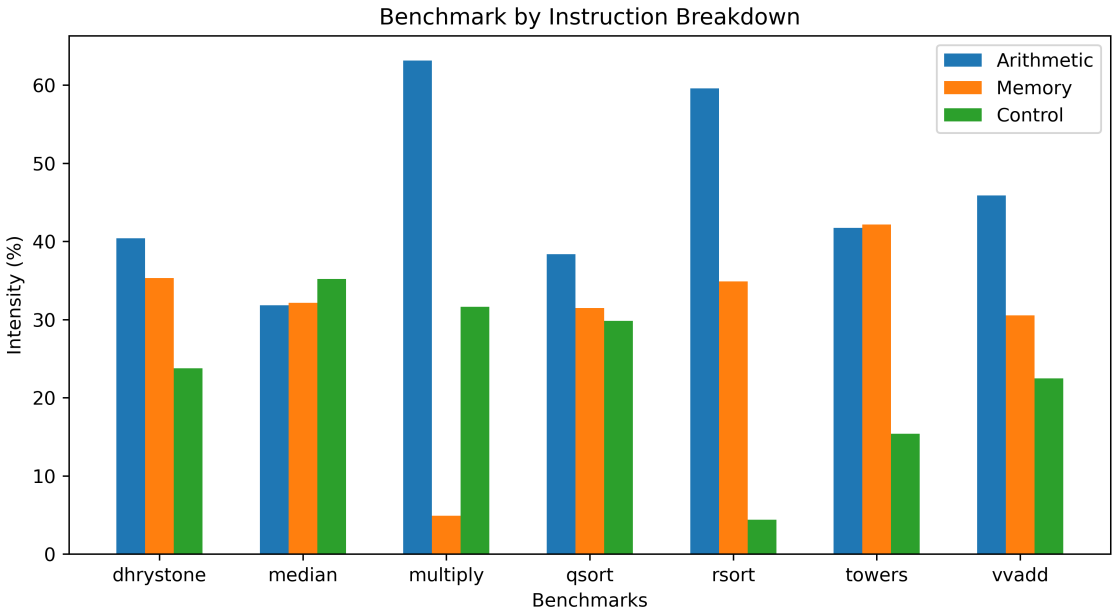
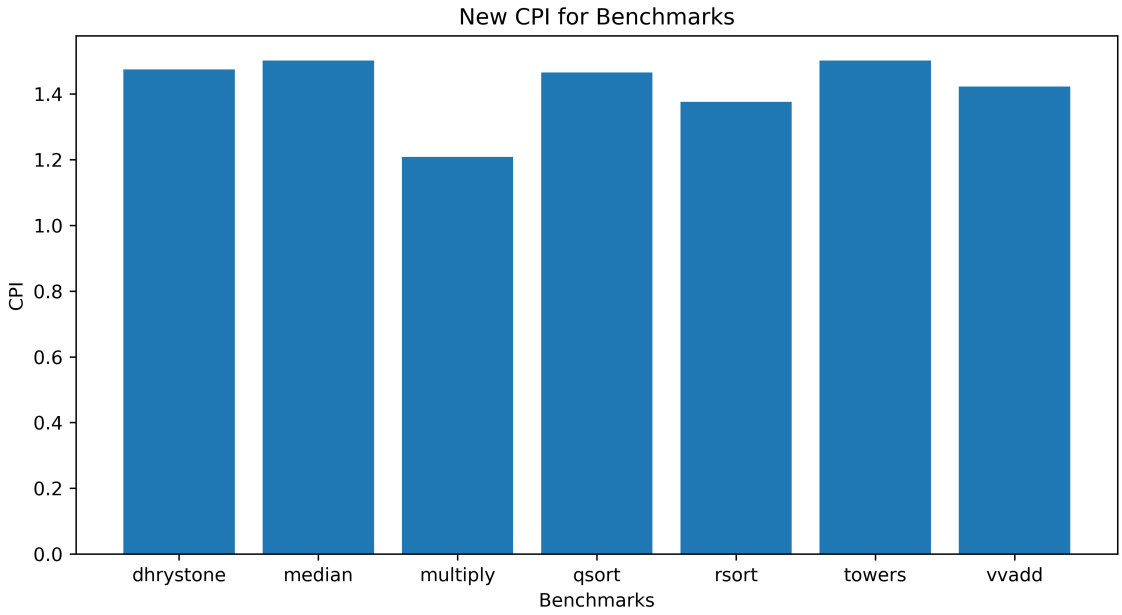
(Directed) Problem 3.4



The benchmark that has the highest arithmetic intensity is multiply, which makes sense as a lot of arithmetic operations would be happening. The benchmark that seems to be the most memory bound is the Hanoi towers benchmark. This makes sense if the algorithm used is the recursive algorithm, causing a lot of stack frame allocation. Benchmark that seems dependent on branch prediction performance is median. This makes sense as you would have a lot of control statements dependent on sorting. The quicksort benchmark also shows similar intensities per instruction due to this.

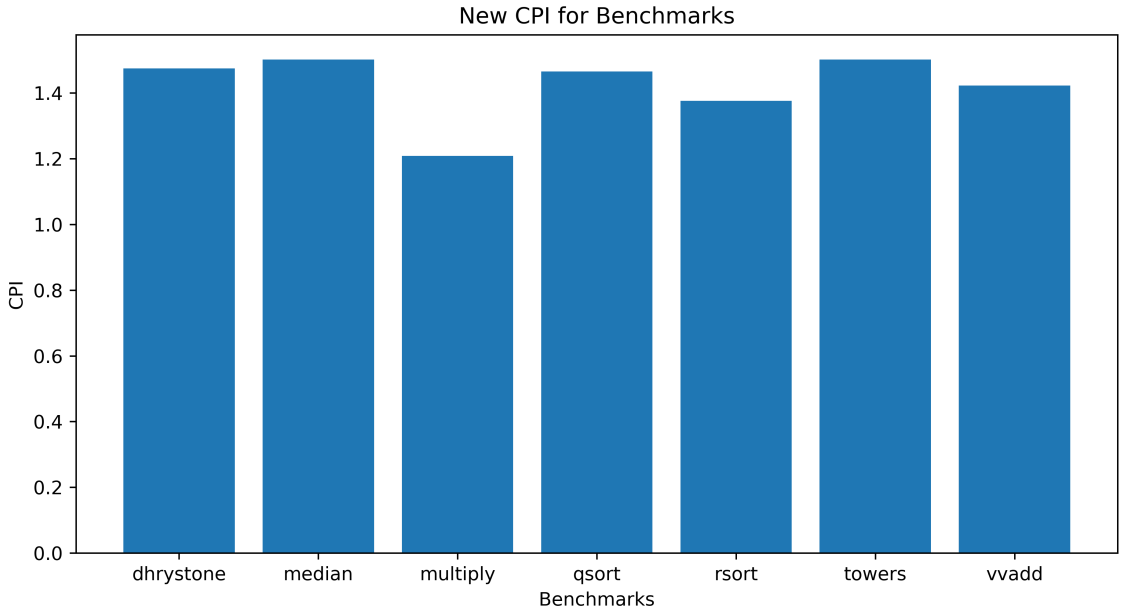
(Directed) Problem 3.5



The overall CPI of the machine for each benchmark is as follows:

Table 1: New CPI for Benchmarks

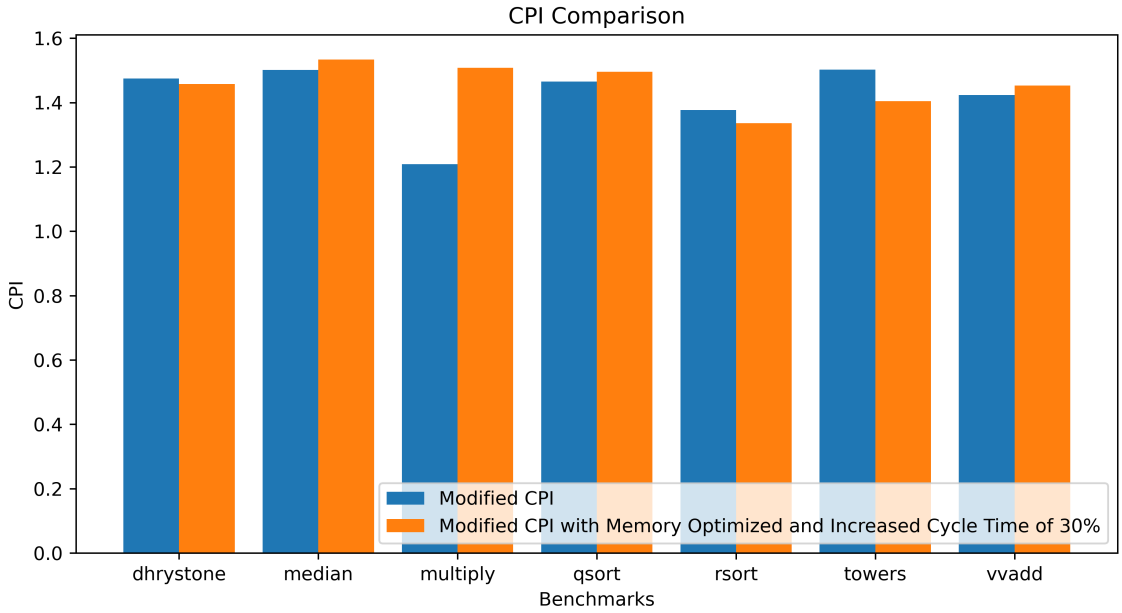
Benchmark	CPI#1
dhrystone	1.475
median	1.501
multiply	1.209
qsort	1.465
rsort	1.376
towers	1.502
vvadd	1.423



For the case that loads and stores are sped up to have an average CPI of 1, then the new CPIs are as follows:

Table 2: New CPI for Benchmarks

Benchmark	CPI#1
dhrystone	1.121
median	1.18
multiply	1.16
qsort	1.151
rsort	1.028
towers	1.08
vvadd	1.118



The plot above shows the comparison of CPIs between the originally given problem prompt and the one optimized with loads and stores. We see that this is an improvements for benchmarks like dhrystone, rsort, and towers. Unfortunately, there is not much of an improvement else where. For just a small speedup, it's not that worthwhile for all the benchmarks. Because there is not one dominating category of instructions for each of the benchmark, the modifications get diluted down in terms of total CPI.

(Directed) Problem 3.6

(Directed) Problem 3.7

(Open-ended) Problem 4.1

(Open-ended) Problem 4.2

(Open-ended) Problem 4.3

(Open-ended) Problem 4.4

(Directed) Problem 5